

SPARK-V: Small Power-Efficient Architecture for RISC-V

1st Justin Lebeau

Electrical and Computer Engineering

Rice University

Houston, Texas, United States

jhl6@rice.edu

Abstract—Wearable biomedical devices require processors that balance performance with strict energy constraints, as battery life directly impacts usability and reliability in real-world health monitoring. Current solutions often rely on proprietary low-power processors, which can be expensive and lack flexibility for customization. Our project proposes an open-source alternative: developing and validating a low-power RISC-V processor tailored for biomedical applications. We begin by validating an existing RISC-V core on an FPGA platform to demonstrate full functionality and suitability for wearable workloads. Building on this foundation, we target a 30% reduction in system power through architectural and implementation-level optimizations. Ultimately, this project aims to produce a validated, energy efficient processor ready for tapeout, enabling more accessible and customizable solutions for biomedical wearables.

Index Terms—RISC-V, Low-Power Design

I. INTRODUCTION

The design goal of this project is to create a processor for use in wearable biomedical devices. Since these devices are battery-powered, they require an architecture that is as low-energy as possible to maximize device usability and reliability.

While low-power processors currently exist, many of these are proprietary, which often makes them restrictive, expensive, or challenging to customize. To address this, the SPARK-V project proposes an open-source alternative using the RISC-V instruction set. The project begins by utilizing WALLY, an existing open-source RISC-V core from Harvey Mudd College, as a baseline [1].

The primary goal is to achieve significant power efficiency, specifically targeting a 30% reduction in system power. This will be accomplished through systematic architectural and implementation-level power optimizations, culminating in a validated, energy-efficient processor.

II. METHODS

This project has been broken across two semesters, and thus I designed a set of goals for each semester. This semester, my first goal is to run Design Compiler for each potential configuration I identified last semester (RV32 IC, IMC, and IC_Zmmul). Using Design Compiler allows each of these configurations to be synthesized, and for evaluation of their power, performance, and area. My second goal is to load each configuration onto an FPGA, and use the Embench test suite to test power usage of each configuration. Finally, I want to

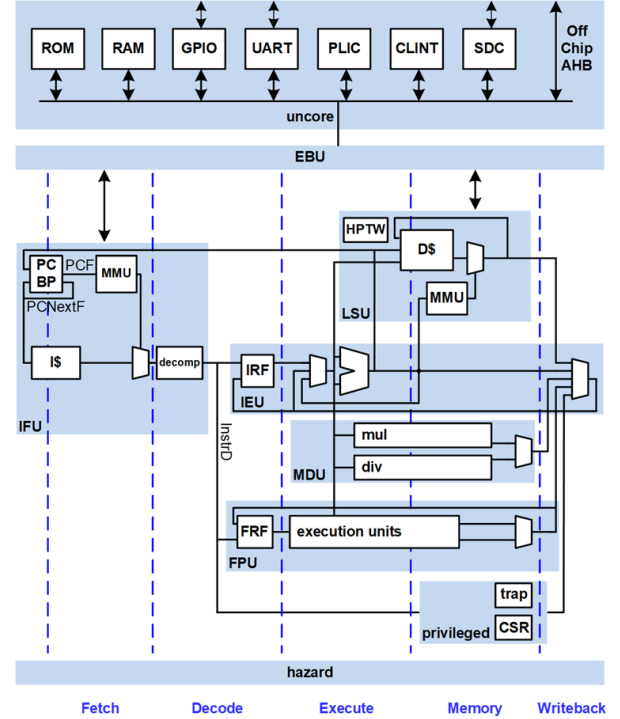


Fig. 1. Block Diagram of the WALLY system [1]

attempt to clock or input gate the multiplication and division unit of the processor to improve power usage.

III. RESULTS

A. Synthesis Using Design Compiler

The initial phase of this project focused on utilizing Synopsys Design Compiler to synthesize three distinct configurations of the WALLY processor to establish an architectural baseline: RV32IC, RV32IC_Zmmul, and RV32IMC. In addition, the RV32GC configuration was synthesized to serve as a comparison point. For this analysis, I used the Sky130 standard cell library. Although this 130 nm technology node is legacy hardware by modern standards, it provides a stable and consistent gate-level environment to evaluate the relative overhead of different RISC-V extensions, and is widely available. By maintaining a uniform timing constraint of a 25 MHz clock

across all synthesis runs, I ensured that any fluctuations in the results were directly attributable to the architectural complexity of the extensions rather than variations in the physical synthesis flow.

The synthesis process yielded specific data points for total cell area, static leakage power, and dynamic switching power for each of the four configurations. These quantitative results now serve as the primary baseline against which the future clock-gating implementations in the multiplication and division unit will be evaluated for power efficiency improvements.

Config	Area (μm^2)	Static Power (nW)	Dynamic Power (mW)	Slack (ns)
RV32IC	261,317	1870	55.3	17.03
RV32IC_Zmmul	306,107	2220	55.36	0.12
RV32IMC	321,394	2330	55.39	-0.04
RV32GC	896,541	5480	57.08	-0.32

Fig. 2. Design Compiler Results

Looking at these results, we can see some trends between the configurations. First, we can examine the three reduced configurations to see area, power, and timing.

For the reduced configurations, we see that dynamic power is relatively consistent, with only a modest rise as complexity increases. However, static power shows a clearer increase with complexity. By including the MDU for multiplication only, the static power increases by 19%, and when the MDU also does division, static power increases by 24%. This correlates almost exactly to the area increase in these configurations, which is expected as static (leakage) power generally scales with area under similar cell libraries and operating conditions. Therefore, we can reduce static power by reducing processor complexity. Since switching activity was not workload-driven and the clock frequency was fixed, dynamic power differences are likely underestimated. It is important to remember this is only an estimate, and more accurate data could be obtained by using real programs in simulation.

The timing reports also vary drastically between the reduced configurations. As previously mentioned, all three designs were synthesized at a 25 MHz clock frequency, which corresponds to a 40 ns clock period. We can see the RV32IC configuration meets this with more than 17 ns to spare, meaning the clock frequency could be greatly increased for this configuration, or that lower power could be obtained by using a reduced voltage. However, the Zmmul configuration barely meets timing, and the IMC configuration has a small negative slack, indicating that the design fails timing at 25 MHz and would require either frequency reduction or optimization.

Comparing the reduced configurations to the full RV32GC configuration shows the largest differences. First, we see that the area is approximately three times that of the reduced configurations, translating into a static power draw of the same magnitude. This demonstrates the true value of reducing extensions supported by the processor, as having this small footprint and low static power can help drive down cost and boost battery life, which is the goal of this project. We see that the dynamic power increases modestly, and that like the RV32IMC configuration, we do not quite meet the desired 25

MHz clock frequency requirement. This could potentially be fixed in physical design, or may require some restructuring of the processor.

B. Testing Embench on an FPGA

In order to implement reduced configurations of WALLY on an FPGA, the workflow used on the GitHub required significant modification. As I did last semester, I am using the Arty A7 100T FPGA Board from Digilent as my FPGA platform. This board contains an AMD Artix-7 FPGA, as well as the needed connections for testing, including JTAG and UART support, as well as PMOD connectors that can be used to access an SD card [2].

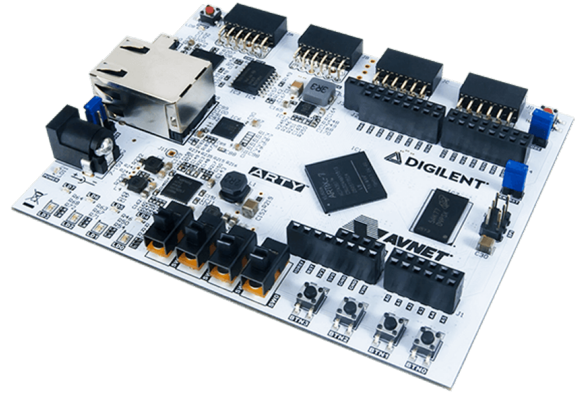


Fig. 3. The Arty A7 100-T FPGA Board [2]

From last semester, I was able to boot linux on the FPGA by synthesizing the RV64gc configuration, and loading the necessary data onto a SD card. Upon power up, the processor would read a program in its bootrom that would instruct it to begin reading from the SD card, booting Linux.

However, for my testing purposes, I wanted to simplify the design to 32-bit configurations, as well as store the program data directly in the processor rather than on external memory.

To start, I modified the Makefile and TCL scripts that invoked Vivado for the FPGA build. I had to make several changes to allow 32-bit compilation, as well as create new configuration files to allow for reduced extension inclusions. Then, I overwrote the bootrom loading process to allow the processor to store my desired program in bootrom rather than the SD card startup code. This has allowed me to add test programs directly into the processor ROM.

To start, I tested one Embench program, crc32, in simulation using Questa. This program computes a cyclic redundancy check (CRC), and its main function and Questa results can be seen in figure 4.

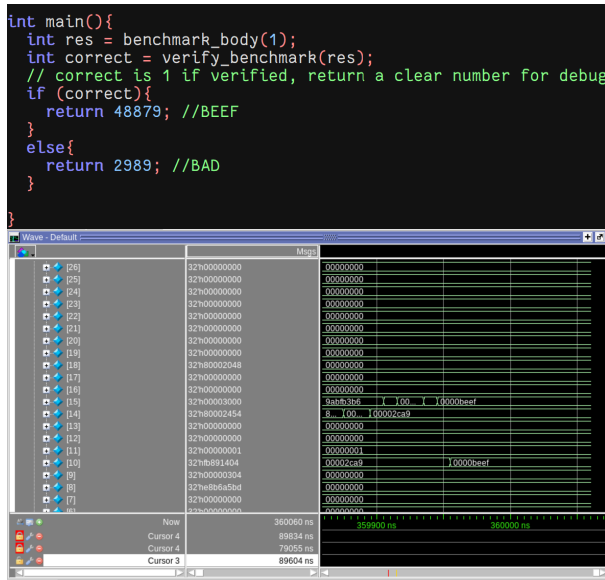


Fig. 4. The `crc32` main function (top), and Questa results showing final register values (bottom).

The main function calls the `benchmark_body` function, which computes the CRC, then calls a function to verify that the final value produced is the expected value. If so, main returns the value 0xbeef into the return register, and if not it returns 0xbad. These values were chosen to aid in debugging. From the Questa waveform, it can be seen that the processor has the correct value in its return register `x10` at the end of program execution, indicating the benchmark simulates correctly.

Getting this program from simulation to running on the FPGA took additional work, including RTL changes. The current commit of WALLY from the Openhw Github (as of April 20th, 2026) only loads actual data into IROM, DTIM, or BOOTROM if the processor is in a 64-bit configuration. In the 32-bit configuration, the RTL directs the processor to load a single line of dummy code to prevent being optimized away. This was done because the FPGA workflow had only been used for 64-bit in the past, and having 2 different loading widths would require additional logic and script control.

This 32 vs 64-bit discrepancy shows up in several other places in the processor, including in the UART RTL. This required some changes to allow loading 32-bit data to the UART transmit FIFO. After making these changes, I was able to verify the program was correctly loading into the Vivado implemented design.

After some more debugging focused on the address map, UART rate, and simulator settings, I was able to successfully run programs on the FPGA and get results back over UART. In order to manage memory sizes, I split Embench into four sections, and loaded them each individually onto the FPGA to run. One section of this can be seen in figure 5.

One important note that can be partially seen in this figure is that not all of the Embench tests were able to run successfully

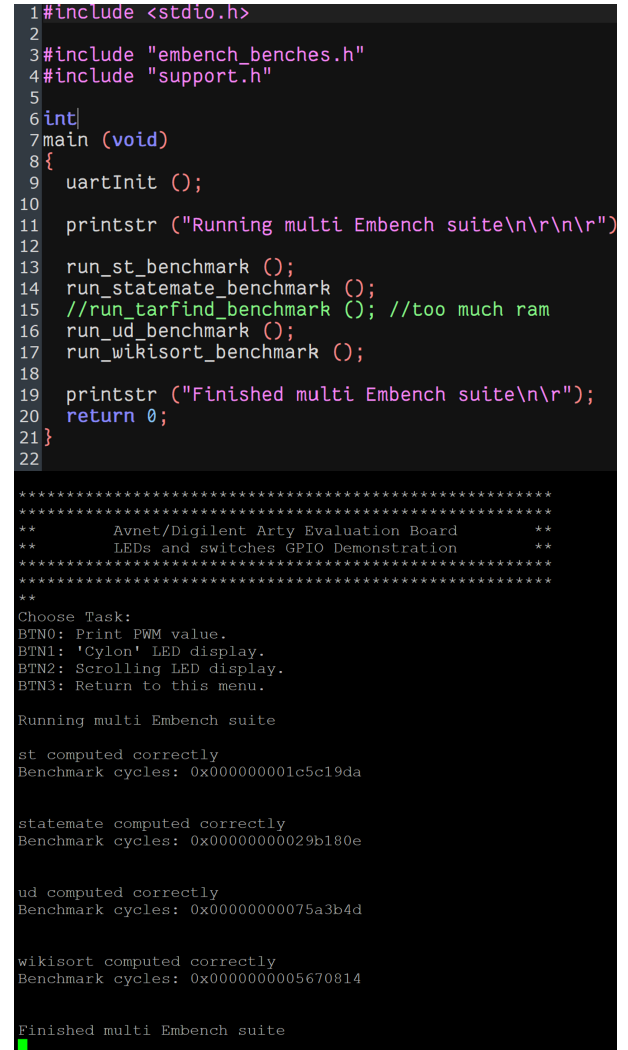


Fig. 5. The `Embench` main function (top), and a screenshot from PuTTY showing results over UART (bottom).

on the FPGA. Of the 22 tests that comprise Embench, 19 ran successfully. The other 3 hung, likely due to memory constraints, though further debugging is required to confirm the root cause. This would be a good place to look for future work.

Each program timed itself using the `mcycle` register. The resulting execution time, measured in clock cycles, was printed back over UART, as can be seen in the figure. These values could then be used to calculate Embench scores using the formula provided in the benchmark specification.

Each of the successful Embench programs were ran using both the `RV32IMC` and `RV32IC_Zmmul` configurations. The resulting scores can be seen in figure 6.

There are some important points that can be learned from this figure alone. First, 11 of the 19 programs showed less than 1% variation from the change. This is a reminder that many embedded workloads simply do not implement division, and therefore division support provides little benefit for these workloads.

Test	IMC Score	IC_Zmmul Score	% Score Decrease
aha-mont64	0.57	0.57	0%
crc32	0.71	0.71	0%
cubic	0.06	0.02	77%
edn	0.67	0.67	0%
matmult-int	0.78	0.78	0%
md5sum	1.31	1.31	0%
minver	0.60	0.21	64%
nbody	0.65	0.13	78%
nettle-aes	0.73	0.68	6%
nettle-sha256	0.71	0.71	0%
nsichneu	0.62	0.62	0%
picojpeg	0.71	0.71	0%
primecount	0.97	0.98	-1%
sglib	1.05	1.04	1%
slre	1.12	1.13	-1%
st	0.79	0.17	78%
statemate	1.83	1.83	0%
ud	0.73	0.64	11%
wikisort	1.47	0.61	58%

Fig. 6. The Embench scores of the 19 successful programs.

Of the remaining 8 programs, 2 showed a minor slowdown of 6% and 11% respectively, and the other 6 showed significant performance reduction with slowdowns between 58% and 78%.

Given these results, it is clear that in division-heavy workloads, having a dedicated divisor unit is essential to performance. Future work on this processor should determine if this is something that is likely to occur and thus should be supported by implementing the full division unit.

We can also compare the results from FPGA emulation with the previous results from Questa simulation. Figure 7 shows the overall scores of the programs.

Emulation (FPGA)

Statistic	IMC Score	IC_Zmmul Score
Geo Mean	0.74	0.52
Geo Std Dev	1.93	2.93

Simulation (Questa)

Statistic	IMC Score	IC_Zmmul Score
Geo Mean	1.07	0.79
Geo Std Dev	1.52	2.07

Fig. 7. Embench summary results from FPGA Emulation (top) and from Questa Simulation (bottom).

The reported scores from Embench are the geometric mean of all program performance scores and the geometric standard deviation of all program performance scores. Comparing emulation and simulation, we see the emulation score is significantly lower than the simulation score.

There are several possible reasons for this, but the most likely is simply the memory structure itself. Due to differences in memory latency such as FPGA BRAM vs idealized simulation, pipeline stalls, and system-level effects not captured in simulation, the programs running on real hardware took a longer time.

Beyond this, there could be additional issues with experimental setup or incorrect assumptions/analysis. Future work should confirm accurate results and analysis are being performed.

C. MDU Clock Gating

Clock gating is a powerful technique to reduce power usage, but its effectiveness varies based on the frequency that a given unit is in use. After discussion with various members of the WALLY project, we agreed that for an integer machine, the majority of functional units are in use at a high enough frequency that the work required to implement fine-grain clock gating would not be worth the effort, and could contradict standard design processes. However, one unit that is often idle is the actual multiplication and division unit (MDU).

I planned to implement either clock or input gating on the MDU to help reduce power consumption. I have attempted this, but ran into issues. This is another direction for future work on this project.

D. Peripherals

WALLY contains a number of peripherals in the "uncore" module. These include ROM, RAM, timers, GPIO, and communication over UART. While the design of the core has been specified above, additional work will be needed to decide which uncore components will be used, and what settings will be applied to them.

Currently, it makes sense to include all of these peripherals to some extent. Program ROM and RAM are fundamental for any standalone system, and the inclusion of PLIC and CLINT enables timer-driven and interrupt-driven execution, both of which are standard requirements for embedded workloads. UART, SDC, and GPIO interfaces also provide practical pathways for off-chip communication, data transfer, and basic system control, making them useful for both development and deployment. In practice, the exact set of peripherals will depend on system constraints. For example, GPIO is flexible and broadly applicable, but the number of exposed pins may need to be reduced if the final processor package has limited I/O availability. Careful selection and scaling of these peripherals will help ensure functionality without exceeding physical or architectural constraints.

IV. DISCUSSION

My work this semester has helped to identify key areas for future work on the SPARK-V processor. Synthesis results using the Sky130 logic library highlight a direct correlation between architectural complexity and power consumption. The transition from a base RV32IC configuration to the RV32IMC, which includes multiplication and division, resulted in a 24%

increase in static power. This increase is almost entirely driven by the expanded silicon area required for the multiplication and division unit. Comparing both of these processors to the full RV32GC machine shows they are both much smaller and more energy efficient. Therefore, a low-power design should focus on optimizing as simple a processor as will be useful.

I was then able to test Embench on the FPGA. I modified the RTL to allow 32-bit data to be loaded into ROM and RAM, and changed the UART configuration to allow a 32-bit transmit buffer. From this, I was able to run most of the Embench programs on the processor, and extract information about their performance. The performances observed were lower than what were predicted in simulation, potentially indicating modeling inaccuracies or hardware inefficiencies that require further investigation.

Unfortunately, I was not able to clock or input gate the MDU.

This leaves several potential directions for future work. One approach could come at the RTL level, working on clock and input gating for the MDU and potentially other blocks of the design. Another is working to further qualify the FPGA performance, including finding the specific reasons each of the 3 failing programs did not print an output, or understanding exactly why the reported scores were lower than the scores from simulation, and seeing if they can be improved. Beyond this, the processor could be taken through physical design, eventually getting to tapeout of a manufacturable chip.

My work this year began the process of creating a low-power RISC-V processor for biomedical applications, and future work could eventually lead to a manufacturable processor.

ACKNOWLEDGMENT

I would like to thank Dr. Ray Simar for his support on this project, as well as Dr. James Stine and Jacob Pease from Oklahoma State University for their continued guidance. I would also like to thank David Harris and Sarah Harris for granting pre-publication access to their textbook covering the WALLY design which has helped in understanding the processor. Additionally, thanks to the Rice Electrical and Computer Engineering Department for funding this project.

CREDIT AUTHOR STATEMENT

Justin Lebeau: Conceptualization, Investigation, Methodology, Software, Validation, Writing

REFERENCES

- [1] <https://github.com/openhwgroup/cvw>
- [2] <https://digilent.com/shop/arty-a7-100t-artix-7-fpga-development-board/>